RTM View Shell Data Connector

Unified Reporting Guide

Real-time and historical reporting on one data model

SQL Server edition · single-tenant (no TenantId)

For the Customer-Service business owner and the BI / data-warehouse team

Edition EN · SQL Server (single-tenant) · 2026-06-10 · RTM View Shell

Column names, grain and units verified against the RTM schema; SQL is T-SQL for a single-tenant Microsoft SQL Server deployment.

Document revision history

This table lists the revision history of this document.

Version	Date	Summary of changes	Product release ID	Shipped-with (commit/barrier)
1.0	2026-06-10	Initial	RTM-REL-2026.06	(pending push)

Contents

About RTM View Shell Data Connector

Scope of this document

This document describes the on-premises, system-per-user installation — direct access to the RTM Microsoft SQL Server database.

RTM View Shell Data Connector is the licensed component that makes RTM's real-time and historical contact-centre data available to your BI / data-warehouse and Customer-Service teams.

Two delivery models:

- **On-premises, system-per-user installation** — the Connector runs inside your own environment and your BI tools query the RTM database **directly**, using the tables and queries in this guide.
- **Cloud (hosted) deployment** — direct database access is not exposed. Instead the Connector provides an **API Feed** that delivers the same data into your client environment (your perimeter).

Licensing

A RTM View Shell Data Connector license is required to use either delivery model.

How to read this guide

This guide explains how to produce **historical BI reports that match the RTM live picture to the second**. It is written for two readers, in two parts you can read independently:

Reader	Read	What you get
Customer-Service business owner	Part 1 (no SQL)	The concepts: the two lifecycles, status groups, how the wallboard and the report stay in agreement.
BI / data-warehouse engineer	Part 2 (SQL)	The data dictionary, the join model, units, and copy-paste worked queries you can reproduce.

This edition: single-tenant SQL Server

This guide targets a single-tenant Microsoft SQL Server database. All queries are T-SQL: identifiers in [brackets], booleans as bit (compare with = 1 / = 0), schema dbo. **There is no TenantId column** — the database holds one tenant's data.

One sentence to remember

RTM persists the same derived agent states that it shows live. So a correctly written historical query returns the same numbers the real-time grid showed — that agreement is the whole point of this document.

Part 1 — The concepts (for the business owner)

1.1 What RTM does with raw contact-centre events

Your ACD (Twilio, or any other platform) emits a stream of low-level events — a call entered a queue, a worker became available, a reservation was accepted, a call was put on hold. On their own these events do not tell you *what an agent is doing right now* or *how a queue is performing*. RTM turns that raw stream into **two clean, business-meaningful models**:

- **A call lifecycle** — every interaction (call, callback, chat) and what happened to it: did it wait, was it answered, abandoned, transferred; how long it waited and talked.
- **An agent-status lifecycle** — a timeline of every agent's status through the day, where each status belongs to a *status group* (Available, On Phone, Break, …).

1.2 The Twilio gap — and how RTM closes it

Legacy ACDs expose a categorised in-call agent status (Available → Ringing → Talk → Hold → Wrap → Available). **Twilio does not.** In Twilio, **Worker Activity is availability only** — it says whether a worker is generally available, and it does *not* change while a call is in progress. The progress of the call itself lives in a separate stream of Voice and Reservation events.

RTM bridges the two. While an agent is on a call, RTM **derives the agent's in-call status from the bound call's state** — Ringing when reserved, Talk when assigned, Hold when the call is held, Wrap Up during after-call work — and writes that derived status into the agent timeline. That is why an RTM wallboard can show “Talk”, “Hold” and “Wrap Up” for a Twilio agent even though Twilio itself never sent those as a status.

Raw Twilio signal	RTM derived agent status
Reservation created	Ringing
Reservation accepted / call assigned	Talk (On Phone)
Call held (hold event)	Hold
After-call work	Wrap Up
Worker Activity = a break activity	Break (from the activity, not the call)

1.3 The two lifecycles in plain language

The call lifecycle: In Queue → Answered → Talk (possibly with Hold and Wrap Up) → Completed. A call that leaves the queue before an agent answers is **Abandoned**.

The agent-status lifecycle: a continuous timeline. Each entry is one status (e.g. Available, Lunch, Talk, Wrap Up, Offline) with a start and end time. Every status maps to exactly one *status group*, which is how individual reasons roll up into shrinkage, occupancy and availability.

1.4 Status groups — the model that must be exactly right

Every agent status belongs to one of these groups:

- **AVAILABLE** — idle and ready to take work.
- **ONPHONE** — engaged on an interaction (the in-call states live here).
- **BREAK** — lunch, short break, etc.
- **PAPERWORK** — after-call work / back-office.
- **TRAINING** — coaching, e-learning.
- **SIGNOFF** — logged out / Offline.
- **UNAVAILABLE** — logged in but not ready, for reasons not mapped elsewhere.

The grouping rule (important for honest reporting)

A status that is explicitly listed under a group (ONPHONE / PAPERWORK / BREAK / TRAINING / SIGNOFF) **belongs to that group.**

Any status that is not mapped falls back to a default bucket: **AVAILABLE** if the worker is available, otherwise **UNAVAILABLE**.

So AVAILABLE and UNAVAILABLE are *default buckets for unmapped statuses*, not hand-picked reason lists.

SIGNOFF (logged out) is its own explicit group. Do not fold sign-off into UNAVAILABLE — they answer different business questions (“not at the desk” vs “at the desk but not ready”).

Handling time: AHT = Talk + Hold + Wrap Up. The exact state→group membership is **per-tenant configuration** — Part 2 shows the sample tenant's real configuration and the one trap it creates.

1.5 Two ways things roll up to a Business Unit

RTM answers questions at the Business Unit (BU) level along two different paths, because calls and agent time reach a BU differently:

- **Calls** roll up by queue: **Queue → Business Unit** (“how did the Sales queues perform?”).
- **Agent time** rolls up by team: **Agent → Agent Group → Super Group → Business Unit** (“how much break time did the Sales team take?”).

1.6 Why real-time and historical agree

The live grid and the nightly report read the **same persisted, derived states**. RTM does not compute one model for the screen and a different model for history — it stores what it shows. Provided a historical query uses the same status-group rules and the same units, it reproduces the live numbers. Part 2, §Example C and §Example D, proves this on sample data.

Part 2 — The practical guide (for the BI team)

The sample tenant used throughout Part 2

Every worked example below uses one small, internally-consistent dataset so you can reproduce the numbers. The identifiers are:

Single-tenant database (no TenantId). All examples filter `OnDate = '2026-06-09'` (yesterday).

Business Units: **Sales** (BusinessUnitId 10), **Support** (20). Queues: `SALES_VOICE, SALES_CB` → `Sales`; `SUP_VOICE` → `Support`.

Team path: agents `agent.dana, agent.omri` → Agent Group `AG_SALES_1` → Super Group `Sales Team A` (100) → BU Sales (10).

2.0 Getting started in 10 minutes

1. **Connect** to the RTM Microsoft SQL Server database. All reporting tables are in schema `dbo`; this guide writes identifiers in T-SQL [brackets].
2. **Learn the three fact tables**: `RTSData_Interaction` (calls), `RTSData_UserStatus` (today's per-status totals per agent), `RTSData_UserStatusLog` (the agent status timeline — the parity backbone).
3. **Always filter on** `[OnDate]`. This is a **single-tenant** database — there is no TenantId column. OnDate is a **string** date key (not a datetime); compare it as text: `[OnDate] = '2026-06-09'`.
4. **Run your first query** — answered Sales calls yesterday:

```
SELECT count(*) AS answered_calls
FROM [RTSData_Interaction] i
JOIN [NGC_BusinessUnitQueueClassification] qc
  ON qc.[QueueId] = i.[Workgroup]
 AND qc.[ClassificationId] = 'ALL'
JOIN [NGC_BusinessUnit] bu
  ON bu.[BusinessUnitId] = qc.[BusinessUnitId]
WHERE i.[OnDate] = '2026-06-09'
  AND bu.[BusinessUnitName] = 'Sales'
  AND i.[IsAnswered] = 1;
```

2.1 Data dictionary

Units are called out per column. Two facts to internalise before reading: **Workgroup = the queue's external id**, and all NGC joins use **string external keys, never the uniqueidentifier primary keys**. This is the single-tenant SQL Server schema — there is no TenantId column.

SQL Server data types

Types below are the standard SQL Server mapping (uuid → uniqueidentifier, timestamptz → datetimeoffset, text → nvarchar(max), varchar → nvarchar). This edition uses datetimeoffset for UTC-offset fidelity (datetime2 in UTC is an acceptable simpler alternative). Confirm types against your deployed MSSQL schema; the column names, grain and units are fixed.

RTSData_Interaction — call fact (grain: one interaction segment)

Column	Type	Key	Meaning
InteractionId	nvarchar(50)	PK	Interaction id. Part of the primary key with Segment + OnDate + ServerId.
Segment	int	PK	Segment number. One interaction can have several segments (e.g. after a transfer). Count DISTINCT InteractionId to avoid double-counting.
OnDate	nvarchar(50)	PK	Business date as a STRING (e.g. '2026-06-09'). The day key for all daily reporting.
ServerId	nvarchar(50)	PK	Source server / site id.
Workgroup	nvarchar(100)	FK	Queue external id. Join to NGC_Queues.ExternalId / NGC_BusinessUnitQueueClassification.QueueId.
UserId	nvarchar(50)	FK	Agent external id that handled the segment.
ClassificationCode	nvarchar(max)		Optional call classification / disposition code.
InteractionType	nvarchar(50)		'Call', 'Callback', 'Chat'.
CallType	nvarchar(50)		e.g. 'External' / 'Internal'.
Direction	nvarchar(50)		'Incoming' / 'Outgoing'.
IsAnswered	bit		True if the segment was answered by an agent.
IsInQueue	bit		True while waiting in queue (not yet answered).
IsTalk	bit		True while the agent is actively talking (in-progress flag).
IsAbandoned	bit		True if the caller left the queue before being answered.
IsTransferred	bit		True if the segment ended in a transfer.
IsMessaging	bit		True for messaging interactions.
IsCallbackRequest	bit		True if the interaction is a callback request.
TimeInQueue	int		Queue wait time in SECONDS.
TalkTime	int		Talk time in SECONDS.
InQueueDateTime	datetimeoffset		When the interaction entered the queue (UTC).
AnsweredDateTime	datetimeoffset		When the interaction was answered (UTC). Unanswered rows carry a sentinel far-past date.
UpdateTime	datetimeoffset		Last update of the row (UTC).
LastUserId	nvarchar(50)		Last agent on the interaction (after transfers).
LastWorkgroup	nvarchar(100)		Last queue on the interaction.
RemoteAddress	nvarchar(50)		Caller number / remote address.

Column	Type	Key	Meaning
`TimeZone`	nvarchar(10)		Site time zone of OnDate.
`CustomCallData, CustomCallData1..20`	nvarchar(max)		Tenant-defined extension fields (disposition, campaign, etc.).

RTSData_UserStatus — today's status totals (grain: one agent × one status × day)

Column	Type	Key	Meaning
`UserId`	nvarchar(100)	PK/FK	Agent external id. Join to NGC_UserAgentgroup.UserId.
`StatusId`	nvarchar(100)	PK	Status identifier.
`ServerId`	nvarchar(50)	PK	Source server / site.
`OnDate`	nvarchar(50)	PK	Business date STRING.
`StatusName`	nvarchar(100)		Human status name (the state, e.g. 'Talk', 'Hold', 'Wrap Up', 'Lunch').
`StatusGroup`	nvarchar(100)		Denormalised group of this status (AVAILABLE / ONPHONE / BREAK / PAPERWORK / TRAINING / SIGNOFF / UNAVAILABLE).
`TotalDuration`	int		Cumulative time in this status today, in SECONDS.
`MaxDuraction`	int		Longest single stay in this status today, SECONDS. NOTE: the column name is mis-spelled in the database (“MaxDuraction”) — quote it exactly.
`TotalCount`	int		Number of times the agent entered this status today.
`DisplayName`	nvarchar(100)		Agent display name.
`UpdateTime`	datetimeoffset		Last update (UTC).
`TimeZone`	nvarchar(10)		Site time zone.

RTSData_UserStatusLog — the status timeline (grain: one status interval)

Units trap on this table

RTSData_UserStatusLog.Duration is in MILLISECONDS — divide by 1000 to get seconds. By contrast, **RTSData_UserStatus.TotalDuration / MaxDuraction** and **RTSData_Interaction.TimeInQueue / TalkTime** are in **SECONDS**. Mixing them is the #1 reporting bug.

Column	Type	Key	Meaning
`Id`	int	PK	Surrogate identity key.
`UserId`	nvarchar(100)	FK	Agent external id.
`StatusId`	nvarchar(100)		Status identifier (the state for this interval).
`ServerId`	nvarchar(50)		Source server / site.
`OnDate`	nvarchar(50)		Business date STRING.
`StartTime`	datetimeoffset		Interval start (UTC). Use for point-in-time reconstruction.
`EndTime`	datetimeoffset		Interval end (UTC). NULL / open for the current ongoing status.
`Duration`	int		Interval length in MILLISECONDS (see trap above).
`StatusGroup`	nvarchar(50)		Group of this status, denormalised onto the interval.
`UpdateTime`	datetimeoffset		Last update (UTC).
`TimeZone`	nvarchar(10)		Site time zone.

The NGC organisation graph (reference / dimension tables)

Join keys are the string external ids (Workgroup, AgentgroupId, UserId, QueueId, SiteId) and the integer BU / Super Group ids — *not* the uuid surrogate keys on NGC_Queues / NGC_AgentGroups.

NGC_BusinessUnit

Column	Type	Key	Meaning
`BusinessUnitId`	int	PK	BU id (identity). Join target for both roll-up paths.
`BusinessUnitName`	nvarchar(100)		Display name (e.g. 'Sales').
`Description`	nvarchar(max)		Free text.
`SiteId`	nvarchar(50)	FK	Site (NGC_Site.SiteId).
`CreatedDatetime / CreatedBy`	datetimeoffset / nvarchar		Audit.

NGC_Queues

Column	Type	Key	Meaning
Id	uniqueidentifier	PK	Surrogate key (do NOT join on this).
ExternalId	nvarchar(100)	KEY	Queue external id — equals RTSData_Interaction.Workgroup.
Name	nvarchar(200)		Display name.
IsActive	bit		Active flag.

NGC_BusinessUnitQueueClassification (queue → BU map)

Column	Type	Key	Meaning
BusinessUnitId	int	FK	Target BU.
QueueId	nvarchar(100)	KEY	Queue external id (= Interaction.Workgroup).
ClassificationId	nvarchar(100)		Must be 'ALL' for a standard “all calls of this queue” mapping. Always include qc."ClassificationId" = 'ALL' in the join.

NGC_Supergroup

Column	Type	Key	Meaning
SupergroupId	int	PK	Super Group id (identity).
SupergroupName	nvarchar(200)		Display name (e.g. 'Sales Team A').
Description	nvarchar(500)		Free text.

NGC_AgentGroups

Column	Type	Key	Meaning
Id	uniqueidentifier	PK	Surrogate key (do NOT join on this).
ExternalId	nvarchar(100)	KEY	Agent-group external id (= SupergroupAgentgroup.AgentgroupId / UserAgentgroup.AgentgroupId).
Name	nvarchar(200)		Display name.
IsActive	bit		Active flag.

NGC_SupergroupAgentgroup (agent-group → super-group map)

Column	Type	Key	Meaning
Id	int	PK	Surrogate identity.
SupergroupId	int	FK	Super Group.
AgentgroupId	nvarchar(100)	KEY	Agent-group external id.

NGC_BusinessUnitSupergroup (super-group → BU map)

Column	Type	Key	Meaning
BusinessUnitId	int	FK	Business Unit.
SupergroupId	int	FK	Super Group.

NGC_UserAgentgroup (agent → agent-group map)

Column	Type	Key	Meaning
Id	int	PK	Surrogate identity.
UserId	nvarchar(100)	KEY	Agent external id (= RTSData_UserStatus.UserId).
AgentgroupId	nvarchar(100)	KEY	Agent-group external id.

NGC_Site

Column	Type	Key	Meaning
SiteId	nvarchar(50)	PK	Site id (e.g. 'IL').
SiteName	nvarchar(200)		Display name.
TimeZone	nvarchar(10)		Site time zone.
ClearTime	nvarchar(5)		Daily midnight-clear time (HH:MM).

Chat: messaging bodies live in `RTSData_ChatMessage` (MessageId, InteractionId, SegmentId, UserId, MsgDirection, DeliveryStatus, TimeStamp). Out of scope for voice-status reporting; join on InteractionId if you need transcripts.

2.2 The join model — the two roll-up paths

Path A — calls: Workgroup → Queue → BU. ClassificationId = 'ALL'. Single-tenant — no TenantId predicate.

```
FROM [RTSData_Interaction] i
JOIN [NGC_BusinessUnitQueueClassification] qc
       ON qc.[QueueId] = i.[Workgroup]
      AND qc.[ClassificationId] = 'ALL'
JOIN [NGC_BusinessUnit] bu
       ON bu.[BusinessUnitId] = qc.[BusinessUnitId]
```

Path B — agent status: UserId → AgentGroup → SuperGroup → BU.

```
FROM [RTSData_UserStatus] us
JOIN [NGC_UserAgentgroup]       ua ON ua.[UserId]=us.[UserId]
JOIN [NGC_SupergroupAgentgroup] sa ON sa.[AgentgroupId]=ua.[AgentgroupId]
JOIN [NGC_Supergroup]           sg ON sg.[SupergroupId]=sa.[SupergroupId]
JOIN [NGC_BusinessUnitSupergroup] bs ON bs.[SupergroupId]=sg.[SupergroupId]
JOIN [NGC_BusinessUnit]         bu ON bu.[BusinessUnitId]=bs.[BusinessUnitId]
```

2.3 Units and gotchas (read before writing any query)

Gotcha	What it means
T-SQL syntax	Identifiers in [brackets]; booleans are bit — compare with = 1 / = 0, not = true/false; default schema dbo.
Milliseconds vs seconds	UserStatusLog.Duration = MILLISECONDS; UserStatus.TotalDuration/MaxDuraction and Interaction.TimeInQueue/TalkTime = SECONDS. Divide the log by 1000 before mixing.
“MaxDuraction” is mis-spelled	The UserStatus column is literally "MaxDuraction". Quote it exactly or the query errors.
OnDate is a string	nvarchar, not a datetime. Compare as text: [OnDate] = '2026-06-09'.
Workgroup = queue	Interaction.Workgroup is the queue EXTERNAL id, not a uuid. Join to NGC_Queues.ExternalId / QueueClassification.QueueId.
String keys, not PKs	All NGC joins use external string ids (UserId, AgentgroupId, QueueId) and integer BU/SG ids — never the uniqueidentifier surrogate keys.
ClassificationId = 'ALL'	Standard queue→BU mappings carry ClassificationId='ALL'. Omit it and you may match nothing or duplicate.
Segment grain	One interaction can have several segments. Count DISTINCT InteractionId for call volumes.

2.4 Worked status-group rollup (with the real config)

Status→group membership is per-tenant. Here is the **sample tenant's actual configuration** (from the RTM Twilio config). State names are the raw ACD activity names — in this Israeli tenant they are in Hebrew; the English gloss is shown for readability.

Group	Member states (raw)	Gloss
ONPHONE	מצלצל; שיחה נכנסת; שיחה יוצאת; המתנה; תיעוד לאחר שיחה	Ringing; Incoming; Outgoing; Hold; Wrap Up
BREAK	break; הפסקה ארוכה	Break; Long break
PAPERWORK	הדרכה חאט; פעילות אישית חאט; אחר חאט …	Training-chat; Personal-chat; Other-chat …
AVAILABLE (default)	(any unmapped status while available)	idle-ready
UNAVAILABLE (default)	(any unmapped status while not available)	not ready
SIGNOFF	(configure your logged-out / Offline states here)	logged out

The ONPHONE double-count trap (must read)

In this sample tenant the ONPHONE group LIST already contains the **Hold** and **Wrap Up** states. So for such a tenant the **ONPHONE group total = Talk + Hold + Wrap ≈ AHT**, NOT talk-only.

Therefore: compute Talk from the **Talk state**, Hold from the **'Hold' state**, Wrap from the **'Wrap Up' state**, and AHT = sum of those three states. **Do not also add the ONPHONE GROUP total** — that double-counts hold and wrap.

A live metric labelled “Talk” that is built on the ONPHONE *group* will, in such a tenant, actually equal talk+hold+wrap. Reconcile talk-only against the **state**, not the group.

2.5 Worked example A — “How many calls did BU Sales abandon yesterday?”

Sample rows from RTSData_Interaction (OnDate = '2026-06-09'). Only the relevant columns are shown; one row per segment:

Field	CALL-1001	CALL-1002	CALL-1003	CALL-1004
Workgroup	SALES_VOICE	SALES_VOICE	SALES_CB	SUP_VOICE
UserId	agent.dana	(none)	(none)	agent.eli
IsAnswered	1	0	0	0
IsAbandoned	0	1	1	1
TimeInQueue (s)	12	45	30	20
TalkTime (s)	180	0	0	0

CALL-1004 is a Support queue, so it is excluded by the BU filter. Two Sales calls abandoned: CALL-1002 and CALL-1003.

```
SELECT count(DISTINCT i.[InteractionId]) AS sales_abandoned
FROM [RTSData_Interaction] i
JOIN [NGC_BusinessUnitQueueClassification] qc
  ON qc.[QueueId]=i.[Workgroup]
  AND qc.[ClassificationId]='ALL'
JOIN [NGC_BusinessUnit] bu
  ON bu.[BusinessUnitId]=qc.[BusinessUnitId]
WHERE i.[OnDate] = '2026-06-09'
  AND bu.[BusinessUnitName] = 'Sales'
  AND i.[IsAbandoned] = 1;
```

Result

sales_abandoned = 2 (CALL-1002, CALL-1003). CALL-1004 excluded (Support); CALL-1001 excluded (answered).

2.6 Worked example B — “Break-time share per Super Group”

Sample rows from RTSData_UserStatus (OnDate = '2026-06-09'; TotalDuration in SECONDS). Both agents are in Super Group **Sales Team A**:

UserId	StatusGroup	TotalDuration (s)
agent.dana	AVAILABLE	10800
agent.dana	ONPHONE	14400
agent.dana	BREAK	1800
agent.omri	AVAILABLE	7200
agent.omri	ONPHONE	18000
agent.omri	BREAK	3600

Break share = Σ BREAK ÷ Σ all = (1800+3600) ÷ (10800+14400+1800+7200+18000+3600) = 5400 ÷ 55800 = 9.68%.

```
SELECT sg.[SupergroupName],
       round(100.0 * sum(CASE WHEN us.[StatusGroup]='BREAK'
                              THEN us.[TotalDuration] ELSE 0 END)
             / nullif(sum(us.[TotalDuration]),0), 2) AS break_share_pct
FROM [RTSData_UserStatus] us
JOIN [NGC_UserAgentgroup]       ua ON ua.[UserId]=us.[UserId]
JOIN [NGC_SupergroupAgentgroup] sa ON sa.[AgentgroupId]=ua.[AgentgroupId]
JOIN [NGC_Supergroup]           sg ON sg.[SupergroupId]=sa.[SupergroupId]
WHERE us.[OnDate] = '2026-06-09'
GROUP BY sg.[SupergroupName];
```

Result

Sales Team A → break_share_pct = 9.68.

2.7 Worked example C — reproduce Talk / Hold / Wrap / AHT and match the live grid

Sample rows from RTSData_UserStatusLog for agent.dana (OnDate '2026-06-09'). **Duration is in MILLISECONDS:**

StatusId	StatusGroup	Duration (ms)
Talk	ONPHONE	180000
Talk	ONPHONE	240000
Talk	ONPHONE	300000
Hold	ONPHONE	30000
Hold	ONPHONE	45000
Wrap Up	ONPHONE	60000
Wrap Up	ONPHONE	60000
Wrap Up	ONPHONE	90000

By STATE (÷1000): Talk = (180+240+300)k ms = **720 s**; Hold = (30+45)k = **75 s**; Wrap = (60+60+90)k = **210 s**. AHT = 720+75+210 = **1005 s = 16:45** over 3 calls = **335 s (5:35) per call**.

```
SELECT l.[StatusId] AS state,
       sum(l.[Duration])/1000.0 AS seconds   -- Duration is MILLISECONDS
FROM [RTSData_UserStatusLog] l
WHERE l.[OnDate] = '2026-06-09'
  AND l.[UserId] = 'agent.dana'
  AND l.[StatusId] IN ('Talk','Hold','Wrap Up')
GROUP BY l.[StatusId];
```

Parity with the live grid

The same per-state seconds appear in RTSData_UserStatus.TotalDuration (Talk 720, Hold 75, Wrap 210) — the figures the wallboard showed live. **They match because RTM persisted the same derived states it displayed.**

Trap check: the ONPHONE *group* total here = 720+75+210 = **1005 s** (≈ AHT), NOT talk-only. For talk-only use the Talk **state** (720 s), as above.

2.8 Worked example D — point-in-time wallboard reconciliation

Question: who was in what status at **11:30 on 2026-06-09**? Reconstruct it from the timeline (StartTime ≤ t < EndTime). Sample intervals:

UserId	StatusId	StatusGroup	StartTime	EndTime
agent.dana	Talk	ONPHONE	2026-06-09 11:28	2026-06-09 11:33
agent.omri	Break	BREAK	2026-06-09 11:15	2026-06-09 11:45

```
SELECT l.[UserId], l.[StatusId], l.[StatusGroup]
FROM [RTSData_UserStatusLog] l
WHERE l.[OnDate] = '2026-06-09'
  AND l.[StartTime] <= '2026-06-09T11:30:00'
  AND (l.[EndTime] > '2026-06-09T11:30:00' OR l.[EndTime] IS NULL);
```

UserId	StatusId	StatusGroup	StartTime	EndTime

Result

At 11:30 — **agent.dana = Talk (ONPHONE)**, **agent.omri = Break (BREAK)**. This is exactly what the live wallboard showed at 11:30.

2.9 Live metric → historical query map

A starting map from common RTM live metrics to their historical source. The metric catalogue (docs/metrics-catalog.json) is the full reference.

RTM live metric	Source	Historical expression
Cumulative Talk Duration (talk-only)	UserStatusLog / UserStatus	sum over Talk STATE; UserStatusLog.Duration/1000 (s)
Cumulative Hold Duration	UserStatus (state 'Hold')	sum TotalDuration where StatusName='Hold' (s)
Cumulative Wrap Up Duration	UserStatus (state 'Wrap Up')	sum TotalDuration where StatusName='Wrap Up' (s)
Average Handling Duration (AHT)	UserStatusLog states	(Σ Talk+Hold+Wrap states) ÷ call count
% Break of login	UserStatus	Σ BREAK TotalDuration ÷ Σ all TotalDuration
Abandoned calls	Interaction	count DISTINCT InteractionId where IsAbandoned, via queue→BU
Answered calls	Interaction	count where IsAnswered, via queue→BU
Current status (point in time)	UserStatusLog	interval covering t (StartTime ≤ t < EndTime)

2.10 Common mistakes (tied to the gotchas)

- **Mixing ms and seconds** — forgetting that UserStatusLog.Duration is milliseconds. Always /1000 before combining with UserStatus/Interaction seconds.
- **Adding the ONPHONE group to talk-only** — in a tenant whose ONPHONE group includes hold/wrap, the group total ≈ AHT. For talk-only, filter the Talk state.
- **Folding SIGNOFF into UNAVAILABLE** — sign-off is a distinct group. Confirm your tenant maps logged-out states to SIGNOFF; otherwise they fall to the UNAVAILABLE default and your shrinkage is wrong.
- **Dropping ClassificationId = 'ALL'** — the queue→BU join needs it; without it you can match nothing or duplicate.
- **Joining on the uniqueidentifier PKs** — join NGC tables on the string external ids (ExternalId / AgentgroupId / UserId / QueueId) and integer BU/SG ids, not on NGC_Queues.Id / NGC_AgentGroups.Id.
- **Using = true / = false** — SQL Server booleans are bit; compare with = 1 / = 0.
- **Counting segments as calls** — use count(DISTINCT InteractionId) for call volumes.
- **Treating OnDate as a date** — it is varchar; compare as a string.

Appendix A — Confirm with your tenant before go-live

- **StatusGroups membership** — which raw status names map to ONPHONE / BREAK / PAPERWORK / TRAINING / SIGNOFF, and whether ONPHONE includes Hold/Wrap (the double-count trap).
- **SIGNOFF mapping** — confirm logged-out / Offline statuses are mapped to SIGNOFF rather than left to the UNAVAILABLE default.
- **Talk / Hold / Wrap state names** — the exact StatusName values your tenant uses (they may be localised, e.g. Hebrew).
- **Queue→BU and team mappings** — that NGC_BusinessUnitQueueClassification (ClassificationId='ALL') and the agent→group→super-group→BU graph are populated for the BUs you report on.

Verification note: *every column name, grain and unit in this guide was checked against the RTM schema and the RTM Twilio configuration on 2026-06-10; SQL Server data types are the standard mapping — confirm against your deployed schema.*

Real Time Metrics Table

This is the `RTSGrid_Metric` catalogue — every metric RTM computes and streams in real time. **To obtain historical data identical to the live figures, build your SQL filters as shown in the Metric Parameter column** (the full table follows on the landscape pages).

How to read the Metric Parameter column (the filter recipe)

Interaction metrics (Function = InteractionsCount, TalkDurationAvg, WaitDurationAvg, …): the parameter is a boolean expression over `RTSData_Interaction` columns. Translate to SQL: `==` → `=`, `!=` → `<>`, `&&` → `AND`, `||` → `OR`, `!x` → `NOT x` (booleans are bit — compare with = 1 / = 0). Apply it as the WHERE clause, plus your queue→BU and OnDate filters.

Agent-status metrics (Function = TotalStatusGroupDuration/Percent, TotalStatusDuration, UsersInStatusGroupCount, …): the parameter is a **StatusGroup** value (e.g. ONPHONE, BREAK) or a **StatusName** value (e.g. Hold, Wrap Up). Filter `RTSData_UserStatus/UserStatusLog` on StatusGroup (group-level) or StatusName (state-level).

Calc metrics (Function = Calc): the parameter is a formula over OTHER metrics in [brackets] — a ratio or percentage. Build each referenced metric with its own Metric Parameter, then apply the arithmetic.

Empty Metric Parameter = an identity/text or fully-derived metric — no direct row filter.

Example 1 — Abandoned calls (QueueNumAbandonedCalls)

Metric Parameter: `(InteractionType=="Call") && (CallType=="External") && Direction == "Incoming" && IsAbandoned && !IsCallbackRequest`

```
-- QueueNumAbandonedCalls   (Function: InteractionsCount)
SELECT count(DISTINCT i.[InteractionId]) AS abandoned_calls
FROM [RTSData_Interaction] i
WHERE i.[OnDate] = '2026-06-09'
  AND i.[InteractionType]   = 'Call'
  AND i.[CallType]          = 'External'
  AND i.[Direction]         = 'Incoming'
  AND i.[IsAbandoned]       = 1
  AND i.[IsCallbackRequest] = 0;   -- !IsCallbackRequest
-- scope to a BU with the queue->BU join from section 2.2
```

Example 2 — Break-group duration (MonAgentBreakDuration)

Metric Parameter: `BREAK` (a StatusGroup value; Function TotalStatusGroupDuration).

```
-- MonAgentBreakDuration   (Function: TotalStatusGroupDuration)
-- Metric Parameter 'BREAK' = a StatusGroup value
SELECT us.[UserId], sum(us.[TotalDuration]) AS break_seconds   -- SECONDS
FROM [RTSData_UserStatus] us
WHERE us.[OnDate] = '2026-06-09'
  AND us.[StatusGroup] = 'BREAK'
GROUP BY us.[UserId];
```

Example 3 — Short inbound calls < 15s (MonAgentNumberOfInboundCalls15sec)

Metric Parameter: `CallType=="External" && Direction=="Incoming" && TalkTime<15 && (InteractionType=="Call" || InteractionType=="Callback")`

```
-- MonAgentNumberOfInboundCalls15sec   (Function: InteractionsCount)
SELECT count(DISTINCT i.[InteractionId]) AS short_inbound
FROM [RTSData_Interaction] i
WHERE i.[OnDate] = '2026-06-09'
  AND i.[CallType]  = 'External'
  AND i.[Direction] = 'Incoming'
  AND i.[TalkTime]  < 15                       -- seconds
  AND i.[InteractionType] IN ('Call','Callback');
```

Note: *the full catalogue (every active metric and its Metric Parameter) is on the landscape pages that follow.*

Real Time Metrics Table — full catalogue

The complete RTSGrid_Metric catalogue (195 active metrics). The **Metric Parameter** column is the recipe to reproduce each metric historically.

Title	Function	Metric Parameter (historical filter recipe)
Calls per Hour (queue)	CPH	`InteractionType=="Call" && Direction == "Incoming"`
Calls per Hour (agent)	CPH	`InteractionType=="Call" && Direction == "Incoming"`
Base Answered % (of all incoming)	Calc	`[QueueNumIncomingOnlineCalls]==0 ? 0 : ((double)[QueueNumAnsweredCalls]/[QueueNumIncomingOnlineCalls])`
Answered % excl. Callback Requests	Calc	`([QueueNumIncomingOnlineCalls]-[QueueNumCallbackRequests])==0 ? 0 : ((double)[QueueNumAnsweredCalls]/([QueueNumIncomingOnlineCalls]-[QueueNumCallbackRequests])`
Answered % incl. Callback Requests	Calc	`[QueueNumIncomingOnlineCalls]==0 ? 0 : ((double)([QueueNumAnsweredCalls]+[QueueNumCallbackRequests])/[QueueNumIncomingOnlineCalls])`
Answered % incl. Completed Callbacks	Calc	`[QueueNumIncomingOnlineCalls]==0 ? 0 : ((double)([QueueNumAnsweredCalls]+[QueueNumCompletedCallbacks])/[QueueNumIncomingOnlineCalls])`
% Abandoned Callbacks	Calc	`[QueueNumIncomingCompletedCallbacks]==0 ? 0 : ((double)[QueueNumAbandonedCallbacks]/[QueueNumIncomingCompletedCallbacks])`
% Abandoned Calls + Callbacks	Calc	`[QueueNumIncomingCompletedCallsAndCallbacks]==0 ? 0 : ((double)[QueueNumAbandonedCallsAndCallbacks]/[QueueNumIncomingCompletedCallsAndCallbacks])`
% Abandoned Calls	Calc	`[QueueNumIncomingCompletedCalls]==0 ? 0 : ((double)[QueueNumAbandonedCalls]/[QueueNumIncomingCompletedCalls])`
% Abandoned Chats	Calc	`[QueueNumIncomingCompletedChats]==0 ? 0 : ((double)[QueueNumAbandonedChats]/[QueueNumIncomingCompletedChats])`
% Abandoned Digital Interactions	Calc	`[QueueNumIncomingCompletedInteractions]==0 ? 0 : ((double)[QueueNumAbandonedInteractions]/[QueueNumIncomingCompletedInteractions])`
% Answered Callbacks in 120 sec (of answered)	Calc	`[QueueNumAnsweredCallbacks]==0 ? 0 : ((double)[QueueNumAnsweredCallbacks120sec]/[QueueNumAnsweredCallbacks])`
% Answered Callbacks in 120 sec (of incoming)	Calc	`[QueueNumIncomingCompletedCallbacks]==0 ? 0 : ((double)[QueueNumAnsweredCallbacks120sec]/[QueueNumIncomingCompletedCallbacks])`
% Answered Callbacks in 30 sec (of answered)	Calc	`[QueueNumAnsweredCallbacks]==0 ? 0 : ((double)[QueueNumAnsweredCallbacks30sec]/[QueueNumAnsweredCallbacks])`
% Answered Callbacks in 30 sec (of incoming)	Calc	`[QueueNumIncomingCompletedCallbacks]==0 ? 0 : ((double)[QueueNumAnsweredCallbacks30sec]/[QueueNumIncomingCompletedCallbacks])`
% Answered Callbacks in 60 sec (of answered)	Calc	`[QueueNumAnsweredCallbacks]==0 ? 0 : ((double)[QueueNumAnsweredCallbacks60sec]/[QueueNumAnsweredCallbacks])`
% Answered Callbacks in 60 sec (of incoming)	Calc	`[QueueNumIncomingCompletedCallbacks]==0 ? 0 : ((double)[QueueNumAnsweredCallbacks60sec]/[QueueNumIncomingCompletedCallbacks])`
% Answered Callbacks	Calc	`[QueueNumIncomingCompletedCallbacks]==0 ? 0 : ((double)[QueueNumAnsweredCallbacks]/[QueueNumIncomingCompletedCallbacks])`
% Answered Calls in 120 sec (of answered)	Calc	`[QueueNumAnsweredCalls]==0 ? 0 : ((double)[QueueNumAnsweredCalls120sec]/[QueueNumAnsweredCalls])`
% Answered Calls in 120 sec (of incoming)	Calc	`[QueueNumIncomingCompletedCalls]==0 ? 0 : ((double)[QueueNumAnsweredCalls120sec]/[QueueNumIncomingCompletedCalls])`
% Answered Calls in 30 sec (of answered)	Calc	`[QueueNumAnsweredCalls]==0 ? 0 : ((double)[QueueNumAnsweredCalls30sec]/[QueueNumAnsweredCalls])`

Title	Function	Metric Parameter (historical filter recipe)
% Answered Calls in 30 sec (of incoming)	Calc	[QueueNumIncomingCompletedCalls]==0 ? 0 : ((double) [QueueNumAnsweredCalls30sec]/[QueueNumIncomingCompletedCalls])
% Answered Calls in 360 sec (of incoming)	Calc	[QueueNumIncomingCompletedCalls]==0 ? 0 : ((double) [QueueNumAnsweredCalls360sec]/[QueueNumIncomingCompletedCalls])
% Answered Calls in 60 sec (of answered)	Calc	[QueueNumAnsweredCalls]==0 ? 0 : ((double)[QueueNumAnsweredCalls60sec]/[QueueNumAnsweredCalls])
% Answered Calls in 60 sec (of incoming)	Calc	[QueueNumIncomingCompletedCalls]==0 ? 0 : ((double) [QueueNumAnsweredCalls60sec]/[QueueNumIncomingCompletedCalls])
% Answered Calls + Callbacks in 120 sec (of answered)	Calc	[QueueNumAnsweredCallsAndCallbacks]==0 ? 0 : ((double) [QueueNumAnsweredCallsAndCallbacks120sec]/[QueueNumAnsweredCallsAndCallbacks])
% Answered Calls + Callbacks in 120 sec (of incoming)	Calc	[QueueNumIncomingCompletedCallsAndCallbacks]==0 ? 0 : ((double) [QueueNumAnsweredCallsAndCallbacks120sec]/[QueueNumIncomingCompletedCallsAndCallbacks])
% Answered Calls + Callbacks in 30 sec (of answered)	Calc	[QueueNumAnsweredCallsAndCallbacks]==0 ? 0 : ((double) [QueueNumAnsweredCallsAndCallbacks30sec]/[QueueNumAnsweredCallsAndCallbacks])
% Answered Calls + Callbacks in 30 sec (of incoming)	Calc	[QueueNumIncomingCompletedCallsAndCallbacks]==0 ? 0 : ((double) [QueueNumAnsweredCallsAndCallbacks30sec]/[QueueNumIncomingCompletedCallsAndCallbacks])
% Answered Calls + Callbacks in 60 sec (of answered)	Calc	[QueueNumAnsweredCallsAndCallbacks]==0 ? 0 : ((double) [QueueNumAnsweredCallsAndCallbacks60sec]/[QueueNumAnsweredCallsAndCallbacks])
% Answered Calls + Callbacks in 60 sec (of incoming)	Calc	[QueueNumIncomingCompletedCallsAndCallbacks]==0 ? 0 : ((double) [QueueNumAnsweredCallsAndCallbacks60sec]/[QueueNumIncomingCompletedCallsAndCallbacks])
% Answered Calls + Callbacks	Calc	[QueueNumIncomingCompletedCallsAndCallbacks]==0 ? 0 : ((double) [QueueNumAnsweredCallsAndCallbacks]/[QueueNumIncomingCompletedCallsAndCallbacks])
% Answered Calls	Calc	[QueueNumIncomingCompletedCalls]==0 ? 0 : ((double)[QueueNumAnsweredCalls]/[QueueNumIncomingCompletedCalls])
% Answered Chats in 120 sec (of answered)	Calc	[QueueNumAnsweredChats]==0 ? 0 : ((double)[QueueNumAnsweredChats120sec]/[QueueNumAnsweredChats])
% Answered Chats in 120 sec (of incoming)	Calc	[QueueNumIncomingCompletedChats]==0 ? 0 : ((double) [QueueNumAnsweredChats120sec]/[QueueNumIncomingCompletedChats])
% Answered Chats in 30 sec (of answered)	Calc	[QueueNumAnsweredChats]==0 ? 0 : ((double)[QueueNumAnsweredChats30sec]/[QueueNumAnsweredChats])
% Answered Chats in 30 sec (of incoming)	Calc	[QueueNumIncomingCompletedChats]==0 ? 0 : ((double) [QueueNumAnsweredChats30sec]/[QueueNumIncomingCompletedChats])
% Answered Chats in 60 sec (of answered)	Calc	[QueueNumAnsweredChats]==0 ? 0 : ((double)[QueueNumAnsweredChats60sec]/[QueueNumAnsweredChats])
% Answered Chats in 60 sec (of incoming)	Calc	[QueueNumIncomingCompletedChats]==0 ? 0 : ((double) [QueueNumAnsweredChats60sec]/[QueueNumIncomingCompletedChats])
% Answered Chats	Calc	[QueueNumIncomingCompletedChats]==0 ? 0 : ((double)[QueueNumAnsweredChats]/[QueueNumIncomingCompletedChats])
% Answered Digital Interactions in 120 sec (of answered)	Calc	[QueueNumAnsweredInteractions]==0 ? 0 : ((double) [QueueNumAnsweredInteractions120sec]/[QueueNumAnsweredInteractions])
% Answered Digital Interactions in 120 sec (of incoming)	Calc	[QueueNumIncomingCompletedInteractions]==0 ? 0 : ((double) [QueueNumAnsweredInteractions120sec]/[QueueNumIncomingCompletedInteractions])
% Answered Digital Interactions in 30 sec (of answered)	Calc	[QueueNumAnsweredInteractions]==0 ? 0 : ((double) [QueueNumAnsweredInteractions30sec]/[QueueNumAnsweredInteractions])

Title	Function	Metric Parameter (historical filter recipe)
% Answered Digital Interactions in 30 sec (of incoming)	Calc	`[QueueNumIncomingCompletedInteractions]==0 ? 0 : ((double) [QueueNumAnsweredInteractions30sec]/[QueueNumIncomingCompletedInteractions])`
% Answered Digital Interactions in 60 sec (of answered)	Calc	`[QueueNumAnsweredInteractions]==0 ? 0 : ((double) [QueueNumAnsweredInteractions60sec]/[QueueNumAnsweredInteractions])`
% Answered Digital Interactions in 60 sec (of incoming)	Calc	`[QueueNumIncomingCompletedInteractions]==0 ? 0 : ((double) [QueueNumAnsweredInteractions60sec]/[QueueNumIncomingCompletedInteractions])`
% Answered Digital Interactions	Calc	`[QueueNumIncomingCompletedInteractions]==0 ? 0 : ((double) [QueueNumAnsweredInteractions]/[QueueNumIncomingCompletedInteractions])`
Current Login Duration	CurLoginDuration	–
Current Login Time	CurLoginTimestamp	–
Current Incoming Call Duration	CurStatusDuration	`Incoming Ext Call`
Current Status Duration	CurStatusDuration	–
Current Status Group	CurStatusGroup	–
Current Status Group Duration	CurStatusGroupDuration	–
Current Status	CurStatusTitle	–
Login Name	DisplayName	–
First Login Time	FirstLoginTimestamp	–
Active Chats (agent)	InteractionsCount	`InteractionType=="Chat" && IsTalk`
Answered Chats (agent)	InteractionsCount	`InteractionType=="Chat" && Direction=="Incoming" && !IsTalk && !IsInQueue && IsAnswered`
Answered Incoming Calls (agent)	InteractionsCount	`(InteractionType=="Call" || InteractionType=="Callback") && Direction=="Incoming" && IsAnswered && !IsTalk`
Long Calls > 10 min (agent)	InteractionsCount	`CallType=="External" && Direction=="Incoming" && TalkTime>600 && (InteractionType=="Call" || InteractionType=="Callback")`
Short Calls < 15 sec (agent)	InteractionsCount	`CallType=="External" && Direction=="Incoming" && TalkTime<15 && (InteractionType=="Call" || InteractionType=="Callback")`
Incoming Calls — External + Internal (agent)	InteractionsCount	`(CallType=="External" || CallType=="Intercom") && Direction=="Incoming" && (InteractionType=="Call" || InteractionType=="Callback")`
Outbound Calls (agent)	InteractionsCount	`InteractionType=="Call" && CallType=="External" && Direction=="Outgoing"`
Outgoing Callbacks (agent)	InteractionsCount	`InteractionType=="Callback" && Direction=="Outgoing" && IsAnswered`
Incoming Callbacks (agent)	InteractionsCount	`InteractionType=="Callback" && Direction=="Incoming" && IsAnswered && ! IsCallbackRequest`
Answered Calls + Callbacks (group)	InteractionsCount	`CallType=="External" && Direction=="Incoming" && !IsTalk && !IsInQueue && (InteractionType == "Call" || InteractionType == "Callback") && IsAnswered`
Outbound Calls (group)	InteractionsCount	`CallType=="External" && Direction=="Outgoing"`
Abandoned Callbacks	InteractionsCount	`(InteractionType=="Callback") && (CallType=="External") && Direction == "Incoming" && IsAbandoned && !IsCallbackRequest`
Abandoned Calls	InteractionsCount	`(InteractionType=="Call") && (CallType=="External") && Direction == "Incoming" && IsAbandoned && !IsCallbackRequest`
Abandoned Calls + Callbacks	InteractionsCount	`(InteractionType=="Call" || InteractionType=="Callback") && (CallType=="External") && Direction == "Incoming" && IsAbandoned && ! IsCallbackRequest`
Abandoned Chats	InteractionsCount	`(InteractionType=="Chat") && (CallType=="External") && Direction == "Incoming" && IsAbandoned && !IsCallbackRequest`
Abandoned Digital Interactions	InteractionsCount	`(CallType=="External") && Direction == "Incoming" && IsAbandoned && ! IsCallbackRequest && (InteractionType=="Chat" || InteractionType=="email")`

Title	Function	Metric Parameter (historical filter recipe)
Active Callbacks	InteractionsCount	`(InteractionType=="Callback") && (CallType=="External") && Direction == "Incoming" && IsTalk`
Active Calls	InteractionsCount	`(InteractionType=="Call") && (CallType=="External") && Direction == "Incoming" && IsTalk`
Active Calls + Callbacks	InteractionsCount	`CallType=="External" && Direction=="Incoming"&& (InteractionType=="Call" || InteractionType=="Callback")&& IsTalk`
Active Chats	InteractionsCount	`(InteractionType=="Chat") && (CallType=="External") && Direction == "Incoming" && IsTalk`
Active Digital Interactions	InteractionsCount	`(CallType=="External") && Direction == "Incoming" && IsTalk && (InteractionType=="Chat" || InteractionType=="email")`
Answered Callbacks	InteractionsCount	`(InteractionType=="Callback") && (CallType=="External") && Direction == "Incoming" && IsAnswered`
Answered Callbacks in 120 sec	InteractionsCount	`(InteractionType=="Callback") && (CallType=="External") && Direction == "Incoming" && IsAnswered && TimeInQueue<120`
Answered Callbacks in 30 sec	InteractionsCount	`(InteractionType=="Callback") && (CallType=="External") && Direction == "Incoming" && IsAnswered && TimeInQueue<30`
Answered Calls	InteractionsCount	`(InteractionType=="Call") && (CallType=="External") && Direction == "Incoming" && IsAnswered`
Answered Calls in 120 sec	InteractionsCount	`(InteractionType=="Call") && (CallType=="External") && Direction == "Incoming" && IsAnswered && TimeInQueue<120`
Answered Calls in 30 sec	InteractionsCount	`(InteractionType=="Call") && (CallType=="External") && Direction == "Incoming" && IsAnswered && TimeInQueue<30`
Answered Calls in 360 sec	InteractionsCount	`(InteractionType=="Call") && (CallType=="External") && Direction == "Incoming" && IsAnswered && TimeInQueue<360`
Answered Calls in 60 sec	InteractionsCount	`(InteractionType=="Call") && (CallType=="External") && Direction == "Incoming" && IsAnswered && TimeInQueue<60`
Answered Calls + Callbacks	InteractionsCount	`(InteractionType=="Call" || InteractionType=="Callback") && (CallType=="External") && Direction == "Incoming" && IsAnswered`
Answered Calls + Callbacks in 120 sec	InteractionsCount	`(InteractionType=="Call" || InteractionType=="Callback") && (CallType=="External") && Direction == "Incoming" && IsAnswered && TimeInQueue<120`
Answered Calls + Callbacks in 30 sec	InteractionsCount	`(InteractionType=="Call" || InteractionType=="Callback") && (CallType=="External") && Direction == "Incoming" && IsAnswered && TimeInQueue<30`
Answered Calls + Callbacks in 60 sec	InteractionsCount	`(InteractionType=="Call" || InteractionType=="Callback") && (CallType=="External") && Direction == "Incoming" && IsAnswered && TimeInQueue<60`
Answered Chats	InteractionsCount	`(InteractionType=="Chat") && (CallType=="External") && Direction == "Incoming" && IsAnswered`
Answered Chats in 120 sec	InteractionsCount	`(InteractionType=="Chat") && (CallType=="External") && Direction == "Incoming" && IsAnswered && TimeInQueue<120`
Answered Chats in 30 sec	InteractionsCount	`(InteractionType=="Chat") && (CallType=="External") && Direction == "Incoming" && IsAnswered && TimeInQueue<30`
Answered Chats in 60 sec	InteractionsCount	`(InteractionType=="Chat") && (CallType=="External") && Direction == "Incoming" && IsAnswered && TimeInQueue<60`
Answered Digital Interactions	InteractionsCount	`(CallType=="External") && Direction == "Incoming" && IsAnswered && (InteractionType=="Chat" || InteractionType=="email")`
Answered Digital Interactions in 120 sec	InteractionsCount	`(CallType=="External") && Direction == "Incoming" && IsAnswered && TimeInQueue<120 && (InteractionType=="Chat" || InteractionType=="email")`

Title	Function	Metric Parameter (historical filter recipe)
Answered Digital Interactions in 30 sec	InteractionsCount	`(CallType=="External") && Direction == "Incoming" && IsAnswered && TimeInQueue<30 && (InteractionType=="Chat" || InteractionType=="email")`
Answered Digital Interactions in 60 sec	InteractionsCount	`(CallType=="External") && Direction == "Incoming" && IsAnswered && TimeInQueue<60 && (InteractionType=="Chat" || InteractionType=="email")`
Callback Requests	InteractionsCount	`(CallType=="External") && Direction == "Incoming" && IsCallbackRequest`
Completed Callbacks	InteractionsCount	`(InteractionType=="Callback") && (CallType=="External") && Direction == "Outgoing" && IsAnswered`
Incoming Callbacks Today (excl. waiting)	InteractionsCount	`(InteractionType=="Callback") && (CallType=="External") && Direction == "Incoming" && !IsInQueue && !IsCallbackRequest`
Incoming Calls Today (excl. waiting)	InteractionsCount	`(InteractionType=="Call") && (CallType=="External") && Direction == "Incoming" && !IsInQueue && !IsCallbackRequest`
Incoming Calls + Callbacks Today (excl. waiting)	InteractionsCount	`(InteractionType=="Call" || InteractionType=="Callback") && (CallType=="External") && Direction == "Incoming" && !IsInQueue && ! IsCallbackRequest`
Incoming Chats Today (excl. waiting)	InteractionsCount	`(InteractionType=="Chat") && (CallType=="External") && Direction == "Incoming" && !IsInQueue && !IsCallbackRequest`
Incoming Digital Interactions Today (excl. waiting)	InteractionsCount	`(CallType=="External") && Direction == "Incoming" && !IsInQueue && ! IsCallbackRequest && (InteractionType=="Chat" || InteractionType=="email")`
Completed Incoming Digital Interactions	InteractionsCount	`(CallType=="External") && Direction == "Incoming" && !IsTalk && !IsInQueue && ! IsAbandoned && (InteractionType=="Chat" || InteractionType=="email")`
Incoming Callbacks Today (incl. waiting)	InteractionsCount	`(InteractionType=="Callback") && (CallType=="External") && Direction == "Incoming"`
Incoming Calls Today (incl. waiting)	InteractionsCount	`(InteractionType=="Call") && (CallType=="External") && Direction == "Incoming"`
Incoming Calls + Callbacks Today (incl. waiting)	InteractionsCount	`(InteractionType=="Call" || InteractionType=="Callback") && (CallType=="External") && Direction == "Incoming"`
Incoming Chats Today (incl. waiting)	InteractionsCount	`(InteractionType=="Chat") && Direction == "Incoming"`
Chats Today (all)	InteractionsCount	`InteractionType=="Chat"`
Outbound Calls	InteractionsCount	`(InteractionType=="Call") && (CallType=="External") && Direction == "Outgoing"`
Transferred Calls	InteractionsCount	`(InteractionType=="Call") && (CallType=="External") && Direction == "Incoming" && IsTransferred`
Waiting Callbacks	InteractionsCount	`(InteractionType=="Callback") && (CallType=="External") && Direction == "Incoming" && IsInQueue`
Waiting Calls	InteractionsCount	`(InteractionType=="Call") && (CallType=="External") && Direction == "Incoming" && IsInQueue`
Waiting Calls + Callbacks	InteractionsCount	`(InteractionType=="Call" || InteractionType=="Callback") && (CallType=="External") && Direction == "Incoming" && IsInQueue`
Internal Calls (agent)	InteractionsCount	`CallType=="Intercom"`
Agents Logged In	LogedInUsersCount	–
Active Interaction ID	LongestInteractionId	–
Customer Phone Number	LongestInteractionRemoteAddress	–
Active Interaction State	LongestInteractionState	–
Active Interaction State Duration	LongestInteractionStateDuration	–
Active Interaction Type	LongestInteractionType	–
Active Interaction Queue	LongestInteractionWorkgroup	–

Title	Function	Metric Parameter (historical filter recipe)
Avg First Response Time — messages (agent)	MessagesAvgFirstResponseTime	`Direction=="Incoming"`
Avg First Response Time (messages)	MessagesAvgFirstResponseTime	`Direction=="Incoming"`
Avg Response Time — messages (agent)	MessagesAvgResponseTime	`Direction=="Incoming"`
Avg Response Time (messages)	MessagesAvgResponseTime	`Direction=="Incoming"`
Max First Response Time (messages)	MessagesMaxFirstResponseTime	`Direction=="Incoming"`
Waiting Chats	NumWaitings	`(InteractionType=="Chat") && (CallType=="External") && Direction == "Incoming"`
Waiting Digital Interactions	NumWaitings	`(CallType=="External") && Direction == "Incoming" && (InteractionType=="Chat" || InteractionType=="email")`
Station ID	Station	–
Average Incoming Call Duration (agent)	TalkDurationAvg	`CallType=="External" && (InteractionType=="Call" || InteractionType=="Callback") && Direction=="Incoming" && !IsTalk`
Avg Talk Duration — Calls+Callbacks (group)	TalkDurationAvg	`CallType=="External" && Direction=="Incoming" && (InteractionType == "Call" || InteractionType == "Callback") && !IsTalk && !IsInQueue`
Avg Talk Duration — Chats (group)	TalkDurationAvg	`CallType=="External" && Direction=="Incoming" && (InteractionType == "Chat") && !IsTalk && !IsInQueue`
Average Talk Duration — Callbacks	TalkDurationAvg	`(InteractionType=="Callback") && (CallType=="External") && Direction == "Incoming" && !IsTalk && !IsInQueue`
Average Talk Duration — Calls	TalkDurationAvg	`(InteractionType=="Call") && (CallType=="External") && Direction == "Incoming" && !IsTalk && !IsInQueue`
Average Talk Duration — Calls + Callbacks	TalkDurationAvg	`(InteractionType=="Call" || InteractionType=="Callback") && (CallType=="External") && Direction == "Incoming" && !IsTalk && !IsInQueue`
Average Talk Duration — Chats	TalkDurationAvg	`(InteractionType=="Chat") && (CallType=="External") && Direction == "Incoming" && !IsTalk && !IsInQueue`
Average Talk Duration — Digital Interactions	TalkDurationAvg	`(CallType=="External") && Direction == "Incoming" && !IsTalk && !IsInQueue`
Longest Current Call (group)	TalkDurationCurMax	`CallType=="External" && Direction=="Incoming"`
Max Call Duration (agent)	TalkDurationMax	`CallType=="External" && (InteractionType=="Call" || InteractionType=="Callback") && Direction=="Incoming"`
Cumulative Login Duration	TotalLoginDuration	–
Consultation Calls (agent)	TotalStatusCount	`Consulting Call`
Missed Calls (agent)	TotalStatusCount	`Missed Call`
Cumulative Incoming Call Duration	TotalStatusDuration	`Incoming Ext Call`
Cumulative Hold Duration	TotalStatusDuration	`Hold`
Cumulative Unavailable Duration	TotalStatusDuration	`Unavailable`
Cumulative Wrap Up Duration	TotalStatusDuration	`Wrap Up`
Average Dialer Call Duration (agent)	TotalStatusDurationAvg	`Campaign Call`
Average Outbound Call Duration (agent)	TotalStatusDurationAvg	`Out Ext Call`
Available State Duration	TotalStatusGroupDuration	`AVAILABLE`
Cumulative Break Duration	TotalStatusGroupDuration	`BREAK`
Cumulative Paperwork Duration	TotalStatusGroupDuration	`PAPERWORK`
Cumulative Talk Duration	TotalStatusGroupDuration	`ONPHONE`
Cumulative Unavailable Duration	TotalStatusGroupDuration	`UNAVAILABLE`
Average Handling Duration (agent)	TotalStatusGroupDurationAvg	`ONPHONE`

Title	Function	Metric Parameter (historical filter recipe)
% Available Time of Login	TotalStatusGroupPercent	AVAILABLE
% Break Time of Login	TotalStatusGroupPercent	BREAK
% Paperwork Time of Login	TotalStatusGroupPercent	PAPERWORK
% Talk Time of Login	TotalStatusGroupPercent	ONPHONE
% Training Time of Login	TotalStatusGroupPercent	TRAINING
% Unavailable Time of Login	TotalStatusGroupPercent	UNAVAILABLE
Extension	UserExtension	—
User ID	UserID	—
Agents in "Missed Call" State	UsersInStatusCount	Missed Call
Agents in Wrap Up	UsersInStatusCount	Wrap Up
Agents in "Available" State	UsersInStatusCount	Available
Agents in "Break" State	UsersInStatusCount	Break
Agents in "On Phone" State	UsersInStatusCount	On Phone
Agents in "Paperwork" State	UsersInStatusCount	Paperwork
Agents in "Training" State	UsersInStatusCount	Training
Agents Available (group)	UsersInStatusGroupCount	AVAILABLE
Agents in Break Group	UsersInStatusGroupCount	BREAK
Agents in Paperwork Group	UsersInStatusGroupCount	PAPERWORK
Agents in Unavailable Group	UsersInStatusGroupCount	UNAVAILABLE
Agents On Call	UsersInStatusGroupCount	ONPHONE
Max Current Break Duration (group)	UsersInStatusGroupDurationCurMax	BREAK
% Time in Break Group (group)	UsersInStatusGroupDurationPercent	BREAK
% Time in Paperwork Group (group)	UsersInStatusGroupDurationPercent	PAPERWORK
% Time in Unavailable Group (group)	UsersInStatusGroupDurationPercent	UNAVAILABLE
Average Time to Abandon — Callbacks	WaitDurationAvg	(InteractionType=="Callback") && (CallType=="External") && Direction == "Incoming" && !IsInQueue && IsAbandoned
Average Time to Abandon — Calls	WaitDurationAvg	(InteractionType=="Call") && (CallType=="External") && Direction == "Incoming" && !IsInQueue && IsAbandoned
Average Time to Abandon — Calls + Callbacks	WaitDurationAvg	(InteractionType=="Call" || InteractionType=="Callback") && (CallType=="External") && Direction == "Incoming" && !IsInQueue && IsAbandoned
Average Time to Abandon — Chats	WaitDurationAvg	(InteractionType=="Chat") && (CallType=="External") && Direction == "Incoming" && !IsInQueue && IsAbandoned
Average Time to Abandon — Digital Interactions	WaitDurationAvg	(CallType=="External") && Direction == "Incoming" && !IsInQueue && IsAbandoned && (InteractionType=="Chat" || InteractionType=="email")
Average Wait Time — Callbacks	WaitDurationAvg	(InteractionType=="Callback") && (CallType=="External") && Direction == "Incoming" && !IsInQueue
Average Wait Time — Calls	WaitDurationAvg	(InteractionType=="Call") && (CallType=="External") && Direction == "Incoming" && !IsInQueue
Average Wait Time — Calls + Callbacks	WaitDurationAvg	(InteractionType=="Call" || InteractionType=="Callback") && (CallType=="External") && Direction == "Incoming" && !IsInQueue

Title	Function	Metric Parameter (historical filter recipe)
Average Wait Time — Chats	WaitDurationAvg	`(InteractionType=="Chat") && (CallType=="External") && Direction == "Incoming" && !IsInQueue`
Average Wait Time — Digital Interactions	WaitDurationAvg	`(CallType=="External") && Direction == "Incoming" && !IsInQueue && (InteractionType=="Chat" || InteractionType=="email")`
Current Max Wait Time — Callbacks	WaitDurationCurMax	`(InteractionType=="Callback") && (CallType=="External") && Direction == "Incoming"`
Current Max Wait Time — Calls	WaitDurationCurMax	`(InteractionType=="Call") && (CallType=="External") && Direction == "Incoming"`
Current Max Wait Time — Calls + Callbacks	WaitDurationCurMax	`(InteractionType=="Call" || InteractionType=="Callback") && (CallType=="External") && Direction == "Incoming"`
Current Max Wait Time — Chats	WaitDurationCurMax	`(InteractionType=="Chat") && (CallType=="External") && Direction == "Incoming"`
Current Max Wait Time — Digital Interactions	WaitDurationCurMax	`(CallType=="External") && Direction == "Incoming" && (InteractionType=="Chat" || InteractionType=="email")`